

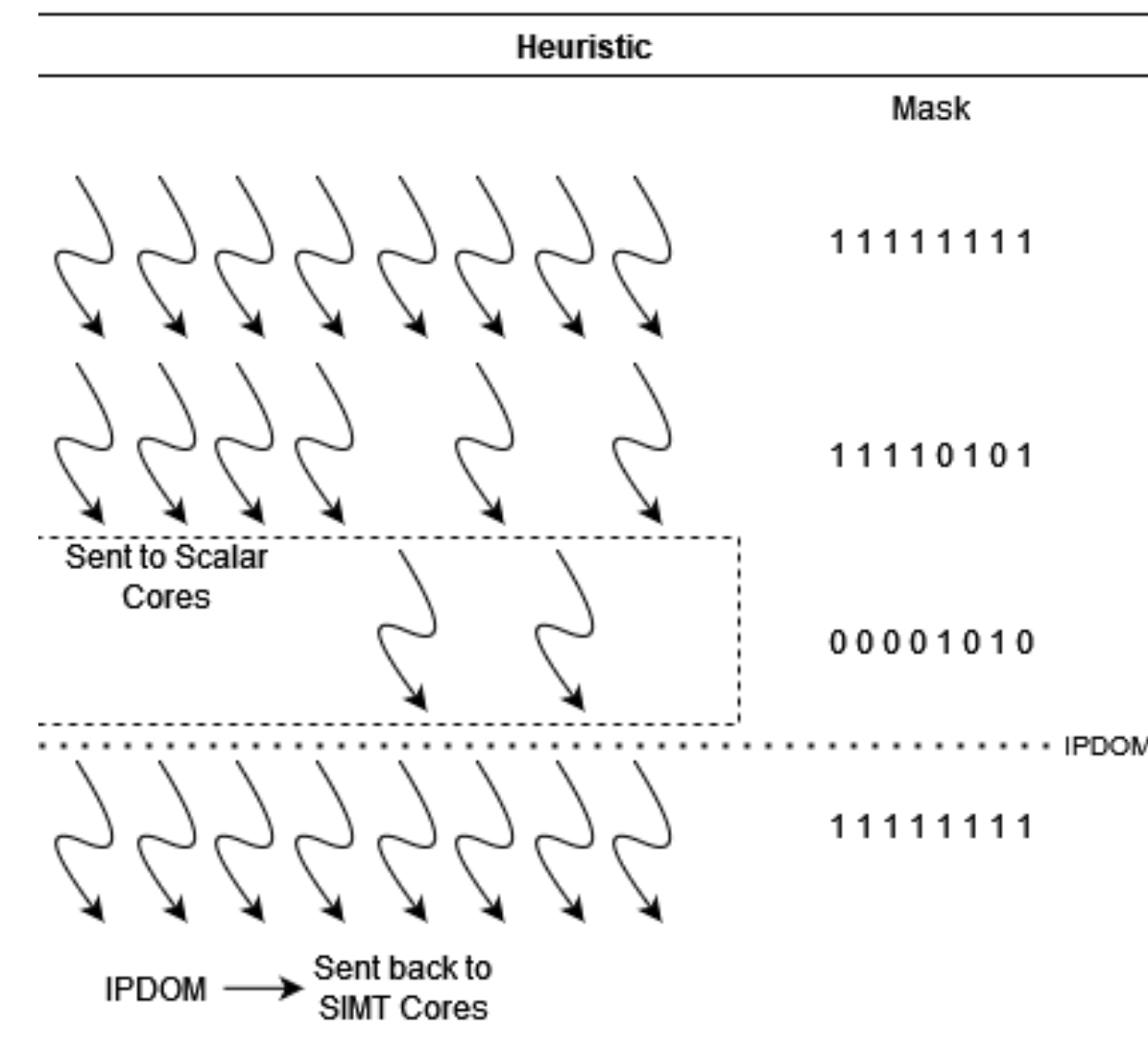
Integrating SIMT and Scalar Cores to Manage Control Flow Divergence OVERVIEW

The main benefit of GPUs, designed with SIMT (single-instruction multiple-data) cores, over CPUs is their high performance per watt. However, this advantage degrades as fewer threads run simultaneously for MIMD (multiple-instruction multiple-data) workloads due to "divergence" and serialized threads. We aim to mitigate this effect by running divergent threads on less power-consuming scalar cores.

We propose two architectures to test the idea. The first relies on software-defined thread prioritization to classify threads based on divergence. It will send the highly divergent threads directly to the scalar core and less divergent threads to the SIMT core. The second architecture utilizes hardware mechanisms to detect and transfer threads from the SIMT core during runtime. This way, the hardware can use intelligent bookkeeping to make decisions about whether threads will diverge.

Control Flow Divergence

- When threads in same warp branch to different paths, execution is serialized
- Leads to low utilization of SIMT pipeline



Implementation

- Implement the architectures on GPGPU-Sim, a cycle-accurate C++ simulator, to achieve a quicker development turnaround time
- Made a lightweight Python simulator to analyze different workloads and get estimated results of our heuristic based on GPU-sim traces.

RTL Implementation

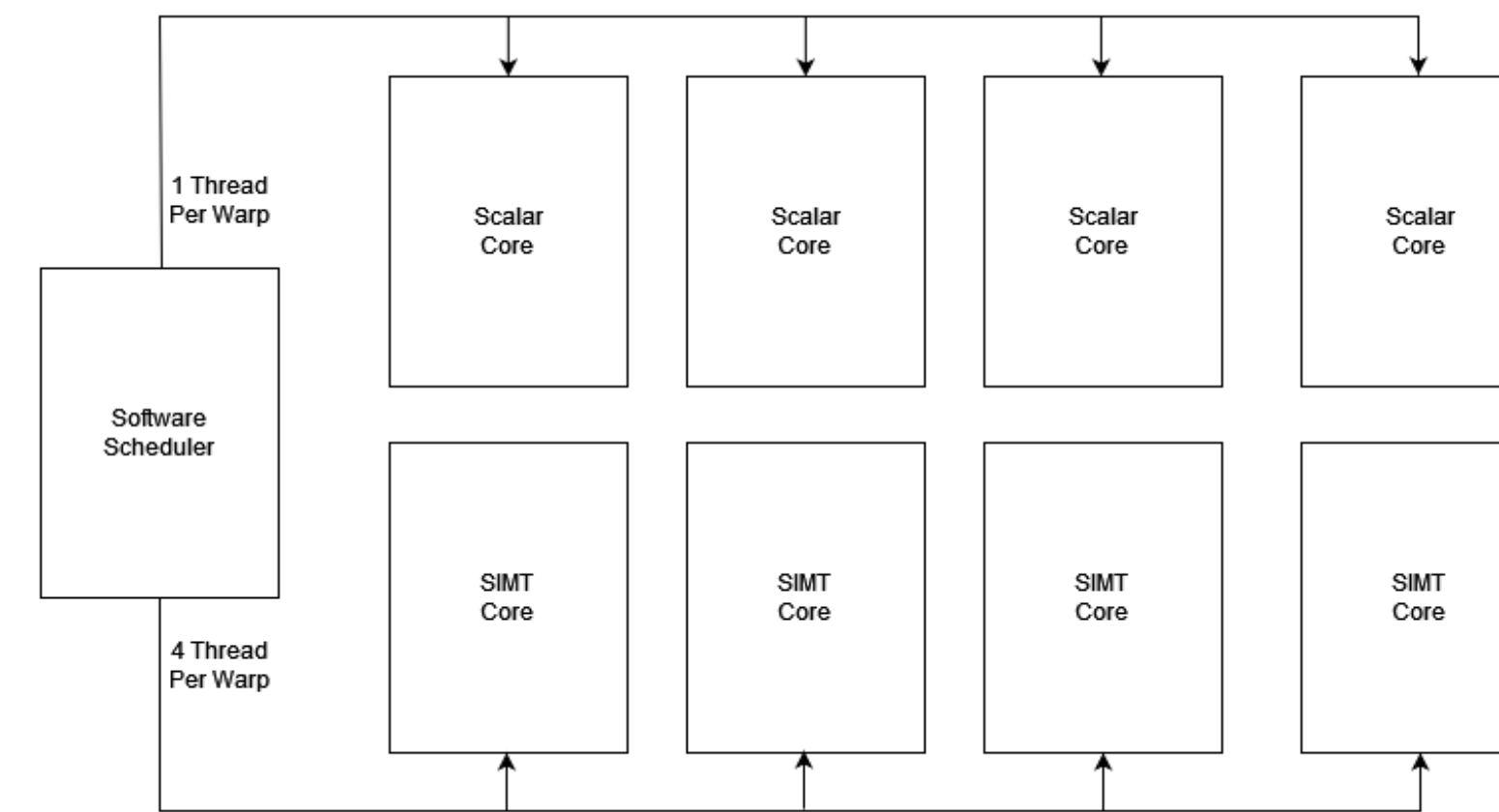
- Developing RTL to ensure hardware-level accuracy for the thread transfer mechanism.
- The dual approach bridges functional evaluation (via GPGPU-Sim) with gate-level feasibility (via RTL), targeting both performance optimization and practical implementation trade-offs.
- Plan to synthesize the design to evaluate power, area, and performance trade-offs against baseline GPUs.

SIMT Parametrization

- To allow for a more general-purpose solution, parameterized the number of threads and warps that could be made for each SIMT core by allowing parameters within each core

Software Defined Transfer

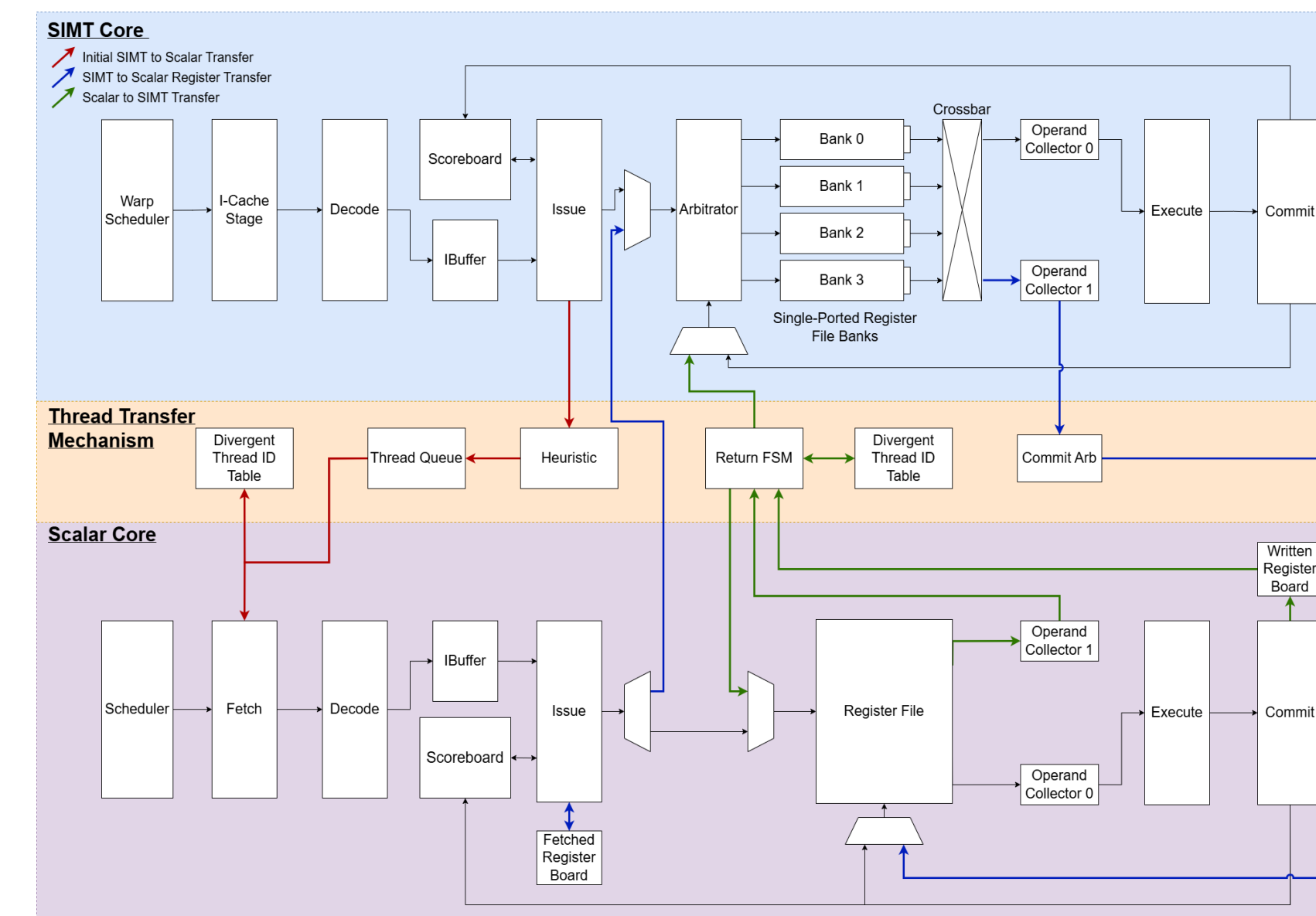
- Allocates a fine-grained multi-threaded "scalar" core for each SIMT core
- Software scheduler will statically assign the scalar threads to the scalar core



Proposed software defined thread prioritization top-level diagram

Hardware Defined Transfer

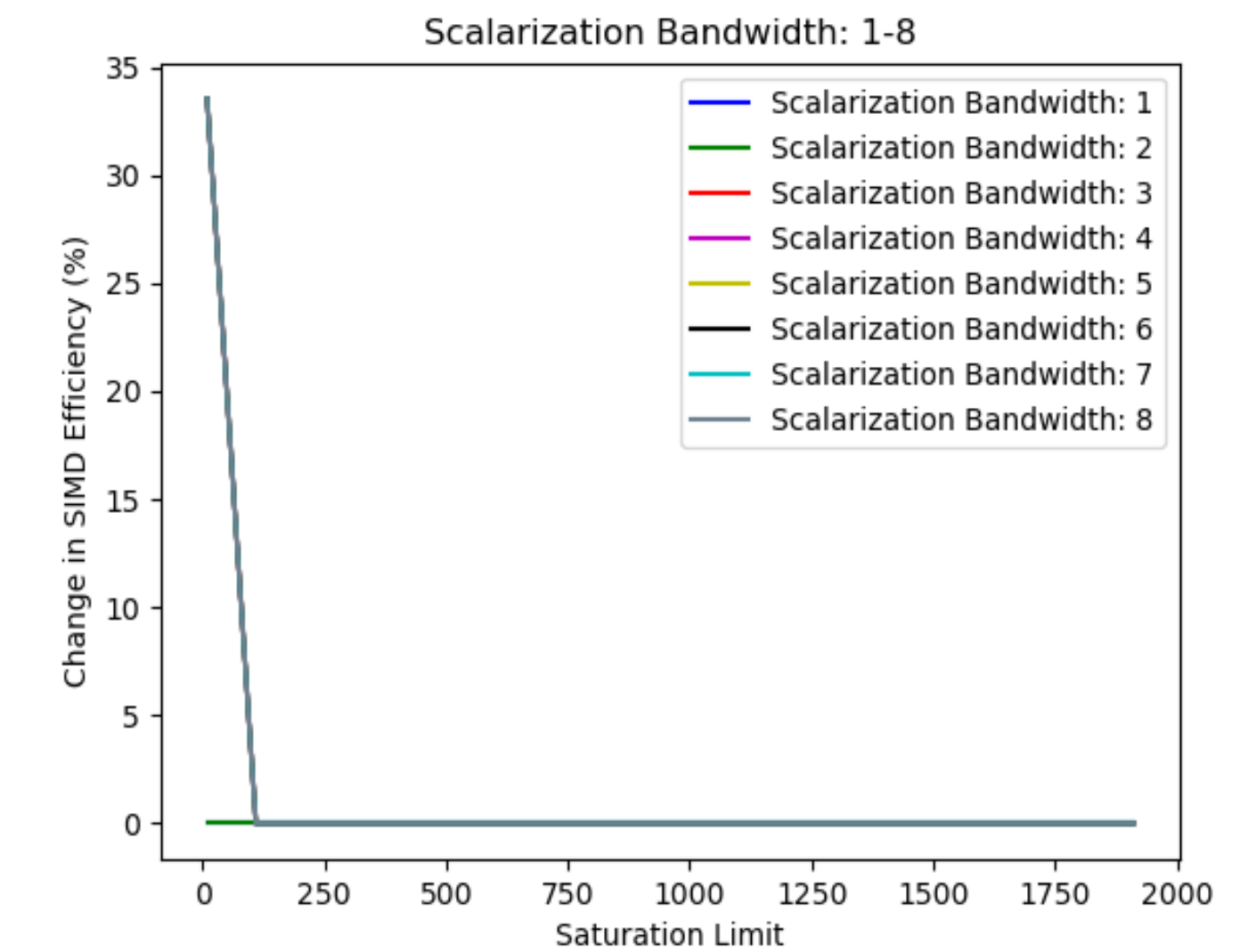
- A more intelligent way of selecting and scheduling heavily divergent threads
- Per-warp hardware heuristic triggers at beginning of the divergent region and finds most divergent sub-group to send to the scalar core
- Threads are pushed to scalar queue then initial thread context is sent to scalar core
- Additional thread context is fetched from SIMT core at on-demand basis
- Once reconvergence point is reached, threads and modified contexts return to the SIMT core



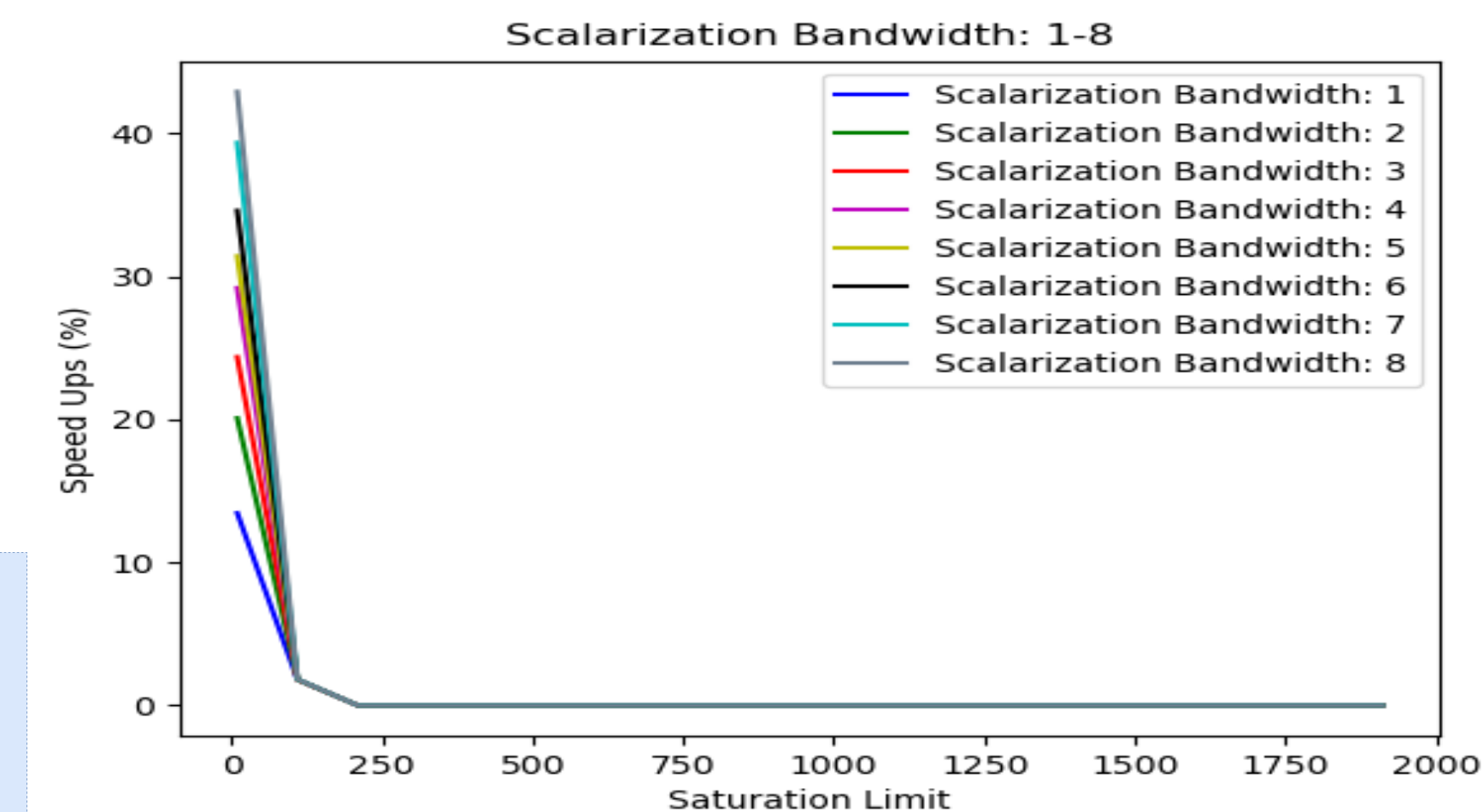
Proposed hardware defined transfer mechanism top-level diagram

Control Flow Analysis

- Performed workload analysis on amorphous parallel algorithms through our Python simulator
- Identified which program would diverge the most by using the total number of divergent masks to calculate the percentage of the divergent program
- Up to 60% total divergence for workloads such as BFS, Gaussian Elimination, and Counting Sort



Gaussian Elimination predicted change in SIMD efficiency on our custom python sim



Breadth First Search predicted speed up on our custom python sim

CONCLUSION

Through our control flow analysis, we identified how algorithms such as BFS, Gaussian Elimination, and Counting Sort can be improved through an integrated SIMT and scalar core architecture. The proposed architectures provide ways to process these algorithms faster and more power efficiently.

CONTRIBUTORS: Hassan Al-alawi, Hunter McCollough, Abhiram Saridena, Khoi Pham, Daniel Choi, Phone Paing, Jerry Chen, Justin Yasuumi, Muhammad Zohaib Ali, Armaan Kanchan, Shrey Mathur

MENTORS: Sooraj Chetput, Dr. Mark Johnson